InChI

InChI = International Chemical Identifier (von IUPAC)

Goal:
- Eindeutiger Identifikator für chem. Stoffe
- Molekül Graph ↔ Identifikator
- Verschiedene Detail Ebenen

Keine z.B. α, β, open chain Isomere
↳ aber dynamische Isomere

Layers = Memoization layer

I) Main Layer: 1. Summenformel, mehrere Komponenten, ...
2. Kovalente Bindung in drei Sub-Layer: zu Schweratom, zu immobilen H-Atomen, zu mobilen H-Atomen

II) Charge Layer: 1. Netto-Ladungen der Komponenten
2. # fehlender / hinzuzufügender Protonen zum neutralen Zustand

III) Stereochemical Layer: 1. E/Z - Isomerie an Doppelbindungen
2. R/S - Isomerie tetra C-Atome

IV) Isotopic Layer: Beschreibung isotopenmarkierter Atome

IV) Fixed-H - Layer: Feste Platzierung einzelner mobiler H-Atome

•, /<char>' : Layer

Main: /{formula} Summenformel
/c{connections} kovalente Bindungen
/h{H-Atoms} Bindung an H-Atome

Charge: /q{charge} Netto Ladungen
/p{protons} Protonen

Stereo: /b{dbond} E/Z
/t{sp3} R/S
/m
/s{stereo-type} // 1 = absolut, 2 = relativ, 3 = racemat

Isotopic: /i

Fixed-H: /f

↳ + optionale Infos in Textform

BRUNNEN

GuCl

InChI-Layer

eg8 InChI= 1S/C9H11NO2/c10-8(9(11)12)6-7-4-2-1/h1-5,8H,6,10H2,(H,11,12)/t8-/m0/s1

Layer 0:

- InChI = <Version> <Normalization-Type> // 1S
  - ↳ welche kanonen Strukturvariationen noch als identisch behandelt werden.

Layer 1 (Main Layer):

Layer 1.1 (Chemical Formula):

- Nach Hill System , , / '
- ~~xxx~~
- falls C: C<#> H<#> [<Elemente nach Alphabet (#)>]
- Sonst: <Elemente nach Alphabet><#>

Layer 1.2 (Connections):

- /c , repräsentiert Bindungsgerüst der Schweratome
- Bindungstypen & Wasserstoffe ignoriert
- Eindeutige Ordnungszahlen:
  - I) Reihenfolge nach Hill system C<N<O
  - II) Gleiche Elemente mit CANON-Algorithmus
- DFS
- Atom-Priorität: <# kovalente Bindung> | <Ordnungszahl> min min
- Bindung , -
- Verzweigung ()
- Ring Atom mehrmals nennen

↳ Bei mehr als 3-nicht H-Bindung über

! 1-a(2,3)q !⇐

BRUNNEN

GrCl

InChI

Layer 1.3 (Hydrogens):

- Einleitung: /h
- H-Klassifikation: mobile: H-Atome, die durch Tautomerie verschoben werden können, alle anderen sind fixed
- < Atome mit 1 H > H1, < Atome mit 2 H > H2, ...
- (< H mobiler H >, < Atome zwischen denen das H verschoben werden kann >)

Layer 3 (Stereochemical), sp^2-Sublayer:

- /b, beschreibt E/Z-Isomerie an Doppelbindungen
- < Atom No 1 > - < Atom No 2 > < +/- >
  - +: Die zu den Atomen 1,2 benachbarten Atome mit höherer Ordnungszahl stehen „Entgegen" (E)
  - -: „... Zusammen" (Z)

Layer 3 (Stereochemical), sp^3-Sublayer:

- /t, /m, /s, beschreibt R/S-Isomerie
- Modi: /s1: absolute: Bild ≠ Spiegelbild verschieden
  /s2: relative: Einige Stereozentren sind bekannt
  /s3: racemat: Gemisch liegt vor
- /t definiert Stereozentren und deren Konfiguration
- < Atom No > < +/- >
  - CIP
  - +/-: Uhrzeigersinn / Gegenuhrzeigersinn
- /m0: InChI entspricht Eingabe, /m1 sonst

BRUNNEN

GaCl

InChI

Layer 2 (Charge):

- /q <Ladungsdifferenz>
- /p <Protonendifferenz>

Layer 4 (Isotopic):

- /i
- Isotopen-Shifts in kanonischer Reihenfolge der Atome
- <Atom-No.><isotopic shift>,...

Layer 5 (Fixed H):

- erlaubt die Festlegung eines speziellen Tautomers

Layer 6 (Fixed Isotopic):

- erlaubt die Festlegung einer speziellen Isotopen-Konfiguration

BRUNNEN

GC1

(5-2) InChI

1) InChI String vs InChI Key

InChI String:
- Der eigentliche InChI
- unique
- variable Länge
- eg: Unique Darstellung eines Moleküls

InChI Key:
- gehashtes InChI String
- nicht unique
- fixed Länge
- eg: Alg die fixed Länge brauchen, Suchmaschinen können sich zu einem gegebenen InChI finden

2) InChI für leere Struktur

InChI = 1S //

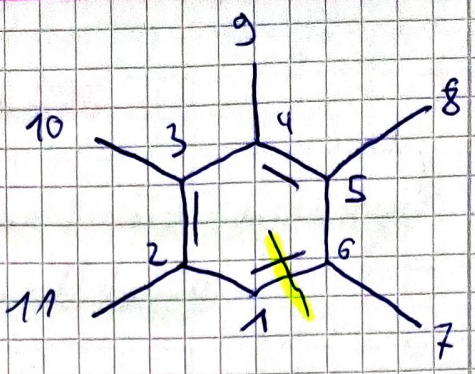
InChI-Key = MOSFIJXAXDLOML-UHFFFAOYNA-N

BRUNNEN

GC1

(S-2)

3)



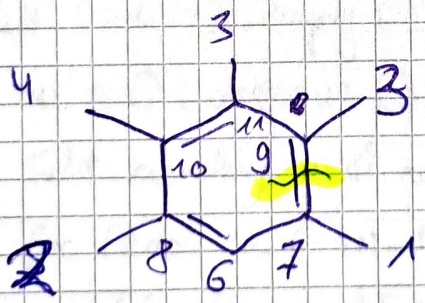
InChI = 1S/C11H16
/c1-2 (3 (4 (5 (6 (1)7)8)9)
10)11)
/h1H, 7-11H3

4) Falsche Kennung → Falsches Ergebnis

5) InChI = 1S/C11H16
/c 1-7-6-8(2)-10(4)11(5)9(7)3
/h 6H, 1-5H3

↳ Unterschied in Connection (1.2) & Hydrogens (1.3)!
Main Layer in

6)



BRUNNEN

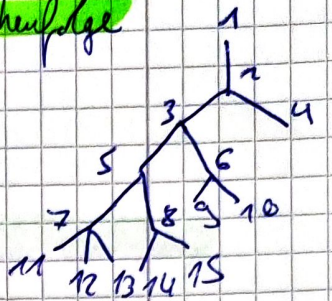
GCI

Kanonische Darstellung chem. Strukturen

Problem: • verschiedene Möglichkeiten für lineare Darstellung ↯
• fehlt eine eindeutige Nomenklatur ↯

Kanonisierung: Erzeuge eine eindeutige, von der Eingabereihenfolge unabhängige, Reihenfolge der Atome

eg: MORGAN, CANON, Molgen-CID



Basis-Alg zur Kanonisierung: BFS

Breitensuche BFS:

↳ systematische Durchmusterung eines Graphen:
• jeder Knoten wird einmal besucht, Knoten werden im Verlauf als besucht (Queue) markiert

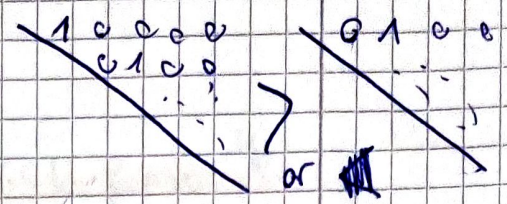
```
BFS (graph G = (V, E))
    for (∀ v ∈ V) queued[v] = False
    for (∀ v ∈ V) if (not queued[v])
        Q.enqueue(v); queued[v] = True;
        while (not Q.empty())
            w = Q.dequeue();
            <do something with node w>
            for (each u ∈ w.adj)
                if (not queued[u])
                    Q.enqueue(u); queued[u] = True;
```

BRUNNEN

GrCI

Morgan-Alg

ego Randic



Naive Lösung:
- → Jede Möglichkeit
- → Textuelle Beschreibung können lexikographisch geordnet werden, wähle die Beschreibung mit niedrigster Ordnung

Reduktion der # der betrachteten Ordnungen:

I) Durchlaufe den Graphen systematisch, erhalte nur Ordnungen, die diesen Durchlauf entsprechen (Morgan-BFS)

II) Definiere eine Ordnung auf den Atomen, diese muss bei der BFS berücksichtigt werden O > N > C

III) Betrachte neben dem Element die # / Art der Bindungen zu Nicht-H-Atomen:

Atomordnung inkl. Element / #Bindungen: O/1 > N/3 > N/2 > C/3 > C/2 > C/1

Einfachb. > Doppelb.

IV) Berechne, connectivity values (CV) für alle Atome durch Einbeziehung der CVs der Nachbar Atome.

Iterative Berechnung:
- ↳ addiere die CVs der adjacenten Atome aus der vorherigen Iteration
- ↳ terminiere, wenn # verschiedener CVs nicht erhöht → elcv[a] key bzgl CV
- ↳ Hält spätestens nach N (# Atomen) Iterationen, da k distinct ist, $1 \le k \le N$

Sumen können zu gleichen Werten führen

Stereo-Kauf wird nicht berücksichtigt

Große Werte

```
CALCULATE-CV (G=(V,E))
    for (∀ v ∈ V) cv[v] = |v.adj|;
    do
        for (∀ v ∈ V) pre_cv[v] = cv[v]
        k = NUMBER-OF-classes (cv);
        for (∀ v ∈ V)
            cv[v] = 0
            for (∀ u ∈ v.adj) cv[v] = cv[v] + pre_cv[v]
    while (k < NUMBER-OF-classes)
```

GC1

Morgan - Alg

=> Es gibt Fälle mit doppelten Element / CV Paaren.

Freiheitsgrade: • Startclan • Reihenfolge Adjazenzliste

Strategie: Aufzählen und über Lösungen minimieren

- Atomreihenfolgen pi mit: elcv[v] < elcv[u] => pi[v] < pi[u] aufzählen
- ∀ Adjazenzlisten werden aufsteigend gemäß pi[v] sortiert
- BFS startend bei pi[s] = 1
- über alle Ordnungen minimieren

I) Cost CVs

Costarien und Ordnung erstellen

Dann bei Max E / Max O starten

und BFS mit Ordnung durchführen

Am ende über Reihenfolge minimieren über Radie

BRUNNEN

GrC I

Morgun-Alg

```
CALCULATE-UNIQUE-ORDER (G=(V,E))
    cv = CALCULATE-CV(G)
    for (∀ v ∈ V) max_order[v] = 1   // initialize arbitrary
    for (∀ atom order pi following elements of cv[] do)
        for (∀ v ∈ V)
            if (pi[v] == 1) s = v;
            v.Adj.sort(pi)   // sort adjacency list cv[]..
        new_order = BFS-CALCULATE-ORDER (G=(V,E), pi, s)
        if (new_order > max_order)   // vgl. actual Adj.-Matrizen
            max_order = new_order
    return (max_order)
```

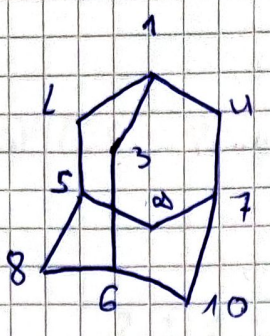
ineffizient alle Reihenfolgen werden aufgezählt und minimal -> Backtracking

```
BFS-CALCULATE-ORDER (G=(V,E), pi, v)
    for (∀ w ∈ V) queued[w] = False;
    Q.init(); Q.enqueue(v); queued[v] = True   // initialize BFS queue
    cur_order[v] = 1; order_no = 1;
    while (not Q.empty())
        u = Q.dequeue()
        order_no += 1
        cur_order[u] = order_no
        for (∀ w ∈ u.adj)   // ordered by ascendy pi values
            if (not queued[w])
                Q.enqueue(w); queued[w] = True;
    return (cur_order)
```

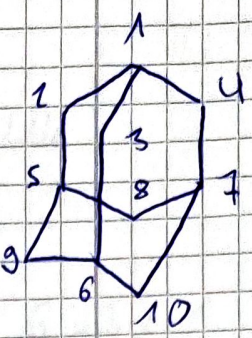
BRUNNEN

G-1 Kanonisierung

1) C/3 > C2



or



h	1	2	3	4	5	6	7	8	9	10	#
0	3	2	2	2	3	3	3	2	2	2	2
1	6	6	6	6	6	6	6	6	6	6	STOPP

	1	2	3	4	5	6	7	8	9	10
1		1	1	1	0	0	0	0	0	0
2			0	0	1	0	0	0	0	0
3				0	0	1	0	0	0	0
4					0	0	1	0	0	0
5						0	0	1	1	0
6							0	1	0	1
7								0	1	1
8									0	0
9										0
10										

1 0 1 0 1 1 <

or

	8	9	10
1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0
5	1	1	0
6	0	1	1
7	1	0	1
8		0	0
9			0
10			

> 0 1 1 1 0 1

Start Atom: O > N > C mit max CV

↳ Adjazenzliste sortieren

4) Nicht identische Moleküle schnell erkennen

Summenformel, Molekülgewicht, Same Formalladung

BRUNNEN

GCI

6-1

2) Morgan - Summen , USMICES - Primzahlprodukte

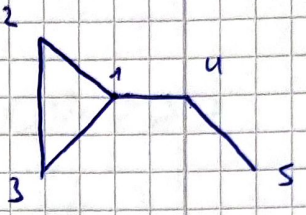
~~[illegible]~~

$2+2+2 = 6 = 3+2+1$, $2 \cdot 2 \cdot 2 = 8 \neq 6 = 3 \cdot 2 \cdot 1$

Wenn Verschiedene Summanden können auf die gleiche Summe kommen. ↯

Ein Primzahlprodukt ist nur gleich wenn alle Primzahlen gleich waren.

3) Kanonische Atomreihenfolge für Ethylcyclopropan

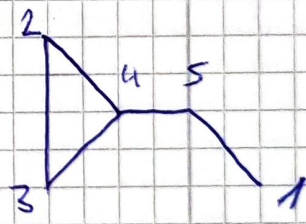


a) Nach Randić-Kriterium

	1	2	3	4	5
1		1	1	1	0
2			1	0	0
3				0	0
4					1
5					

→ maximiert

b) Nach Morgan ohne BFS (min Adjazenz)



[illegible]

	1	2	3	4	5
1		0	0	0	1
2			1	1	0
3				1	0
4					1
5					

BRUNNEN

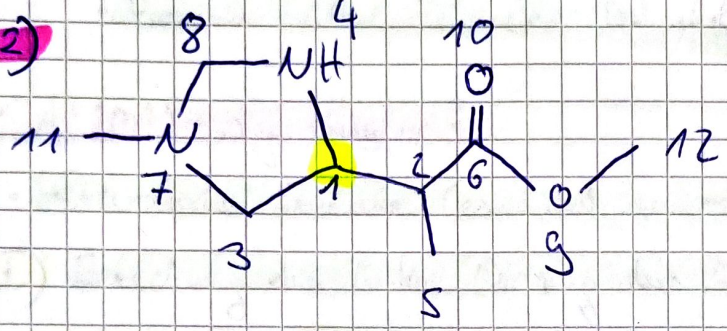
GC1

(6-2) Morgan

1)

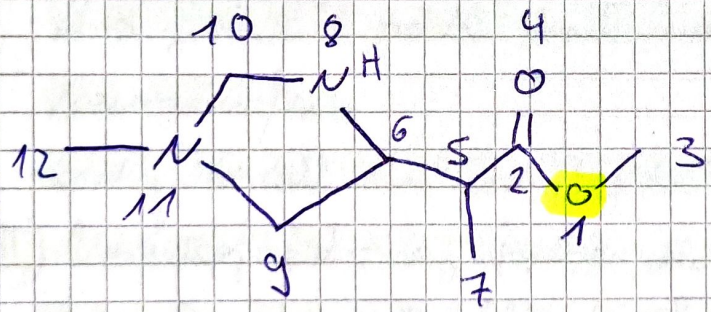
	C	O	C	O	C	C	C	N	C	C	N	C	
	1	2	3	4	5	6	7	8	9	10	11	12	#CVs
0	1	2	3	1	3	1	3	2	2	2	3	1	3
1	2	4	6	3	7	3	7	5	6	5	5	3	6
2	4	8	14	6	16	7	18	12	18	10	14	5	9
3	8	18	36	14	39	16	40	28	32	26	27	14	11
9	18	38	71	30	86	39	99	66	67	55	72	27	12
	C	O	C	O	C	C	C	N	C	C	N	C	

2)

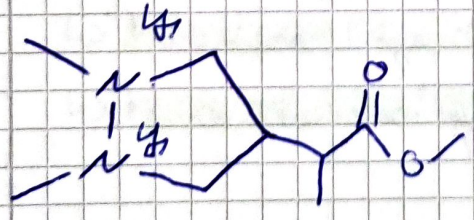


CVs >>>

3) O > N > C dann CVs >>>



4) Keine eindeutige Zuordnung in Morgan



⇒ Lösung: Aufzählen aller möglichen Lösungen die, die CVs berücksichtigen zB über alle Ordnungen maximum !

Radić

BRUNNEN

GCI

Unique SMILES

Problem: Kanonische SMILES für ein Molekül

↳ Freiheitsgrade bei DFS:
- Startatom festlegen
- Reihenfolge der Adjazenzliste
- Auswahl und Numerierung der Ring-Bindungen

Idee:
- Berechne eine eindeutige nur auf dem Molekülgraph basierende Ordnung der Atome (Kanonisierung)
- Prozessiere Atome in dieser Reihenfolge

CANON-Algorithmus:
- Erweiterte Summe (extended sums)

I) Berechnung einer initial Ordnung der Atome:

Knotengrad &, # nicht-H Bindg., (gewichtet mit Bindtype), Element, sgn(Ladung), abs(Ladung), # gebd. H-Atome

- Ableitung einer Zahl pro Atom durch lexikographische Ordnung der Atomeigenschaften

1-1-7-0-0-2
↳ ①

- Ersetzung der Zahl durch ihren Rang im Molekül

II) Verknüpfung der Ranginformation der benachbarten Atome und erzeuge eine neue Rangfolge, bis Konstant.

↳ ersetze Rang durch Primzahl mit gleichem Rang
↳ Produkt der Ränge der Adjazenzatome
↳ unter Berücksichtigung des Alten Ranges neue Ränge

BRUNNEN

GCI

Eindeutige SKLE - CANON

$P_{min} = P_{min}$ if all Faktoren are equal

eg.:

Ordnung	Primzahl	Produkt	Ordnung	Primzahl	Produkt	Ordnung
1	2	13	1	2		1
2	3	17	→ 2	3		2
3	5	17	→ 3	5		3
4	7	221	4	7		4
5	11	121	5	11	121	5
5	11	121	5	11	143	6
5	11	143	5	11	143	6
5	11	187	6	13	253	7
5	11	187	6	13	253	7
6	13	238	7	17		8
7	17	847	9	23		10
7	17	130	8	19		9

III) Erzeugung einer vollständigen Ordnung durch iteratives Tie-Breaking

Tie-Breaking-Operation:

↳ multipliziere alle Ränge mit 2

↳ subtrahiere vom ersten Atom ist doppelt auftretendes Rang 1

↳ generiere initiale Rang Folge → II)

BRUNNEN

GACI

Unique Labels compare

eg:

Ordnung	Canonical Tie Break	Ordnung	Primes	Produkte	Ordnung
1	2	1	2		1
2	4	2	3		2
3	6	3	5		3
4	8	4	7		4
5	10	5	11		5
6	11	6	13		6
6	12	7	17		7
7	14	8	19	527	9
7	14	8	19	403	8
8	16	9	23		10
10	20	11	31		12
9	18	10	29		11

Unique SMILES

GENES-Alg:

-> Basis: 2×DFS

-> Initial-Knoten: Atom mit Ordnungszahl 1

-> Verzweigung an Atom: Nachbarbindung in aufsteigender Ordnung verarbeitet

-> Am Ring system: • Drei > Doppel > Einfachbindung
• Danach Ordnungszahl

-> funktioniert aber formell inkorrekt!

BRUNNEN

GCI

Molgen-CID Kanonisierer

-> exakter Kanonisierer ✓

-> keine Berücksichtigung von Stereozentren

-> keine impliziten H-Atome

-> Atome durch Elementarzahl in kanonischer Reihenfolge

↳ Bindungsliste: <typ><Atom No><typ><Atom No>...

↳ jede Bindung nur bei erstmaligen auftreten listen.

-> Initialordnung:

I) Element gemäß Ordnungszahl: O > N > C

II) Atomeigenschaften: Ladung (≠0), Radikal, etc.

III) Topologie: Ring vor Kette

IV) Kette-Gerüst vor Kette-Nichtgerüst

V) Bindung

-> Iteratives Refinement:

-> Queue alle Atome mit bereits eindeutiger ID

-> Ein Atom a mit eindeutiger ID kann eine Menge S von Atomen mit gleicher ID splitten: $S_1 = \{x \in S \mid x \in a.Adj\}$, $S_2 = S \setminus S_1$

↳ werden in Queue eingefügt.

-> Gerüst für gleiche Ordnung:

-> initiale Ordnung & Refinement keinen Atome nicht trennen

-> automorph ✓

-> Backtracking: Maximum über alle so erstellten Ordnungen (Radic)

↳ kluge Dreiecksmatrix

↳ Speicher Automorphismen

BRUNNEN

GCI

Anwendung Chemische Muster

↳ Partitionierung großer Substanzen

[:]

↳ Beschreibung funktionaler Gruppen

↳ Def. toxische Molekülfragmente

Chemische Strukturen vs. Chemische Muster

Molecules are represented by systematic names,
synonyms,
structure diagrams.

Chemical patterns are represented by chemical pattern language,
visualization with SMARTS viewer.

BRUNNEN

GCI

SMARTS

Sprache zur Beschreibung von chemischen Substrukturen

↳ basiert auf SMILES

↳ erlaubt die Umschreibung von Atomen zu Bindungen nach Eigenschaften

↳ regex

SMARTS = SMILES Arbitrary Target Specification

NOT
AND
OR

- ATOM (elem, chare, nums, chirality) → Properties (valence, aromatic, connectivity, ring)
  → Recursive Specifications (environment of an atom)
- Band (s, d, t, a) → Binding modes (ring, direction, any bond)

- Alles wie bei SMILES plus:
  - `*` : beliebiges Atom
  - `a` : aromatisches Atom
  - `A` : aliphatisches Atom
  - `D<n>` : Atom mit <n> expliziten Bindungen // meist zu nicht H!
  - `X<n>` : Atom mit insgesamt <n> Bindungen
  - `v<n>` : Atom mit Bindungsordnung <n> → eg: C hat 4 -!-
  - `H<n>` : Atom mit <n> benachbarten H-Atomen
  - `h<n>` : Atom mit <n> benachbarten impliziten H-Atomen
  - `x<n>` : Atom mit <n> Bindungen in einem Ring

BRUNNEN FH

GCI

SMARTS

implizit / explizite H-Atome:

-> explizit: H-Atom erscheint in SMILES

-> explizit: H-Atom durch Hydrogen Count

-> implizit: durch Valenzprüfung

Atomeigenschaften:

R<n>	in <n> Ringen eines SSSR
r[n]	in dem kleinsten Ring eines SSSR mit <n> Atomen
-<n>	negative Formalladung <n>
+<n>	positive Formalladung <n>
#<n>	Atom mit Ordnungszahl <n>
@	lokale Chiralität ↻
@@	lokale Chiralität ↺
<n>	mit Masse <n>

Bindung:

	Einfach / Aromatisch
-	Einfach Aliphatisch
/	Einfach Up
\	Einfach Down
/?, \?	Up/Down oder nicht spezifiziert
=	Doppel
#	Dreifach
:	aromatisch
~	beliebig
@	beliebige Bindung im Ring

BRUNNEN

GCI

SMARTS

Logische Ausdrücke:

Stark

!<e>	: nicht <e>
<e1>&<e2>, <e1><e2>	: Und (Stark)
<e1>,<e2>	: ~~Exklusiv~~ Oder
<e1>;<e2>	: Und (Schwach)

Atom ausdrücke können mit logischen Operatoren in [ ]

Rekursion:

Umgebung wird rekursiv beschrieben:

- $(<SMARTS>)
- Das erste Atom des SMARTS beschreibt das zweite zu spezifizierende Atom (Match)
- geht geschachtelt

eg: H_3C-OH matcht COC nicht aber C[$(OC)]

Trennsymbol:

<a1>.<a2>	: können aber müssen nicht verbunden sein
(<SMARTS>)	: keine Wirkung
(<s1>).(<s2>)	: Matchen in verschiedenen Fragmenten
(<s1>).<s2>	: unabhängig von einander

BRUNNEN

GCI

(S-1) SMARTS

1) a) kovalente Halogenverbindungen:

*~[F, Cl, Br, I, At]

↳ 1 C und 1 d C und *

b) heterocyclische Ringsysteme aus 3 drei Atomen

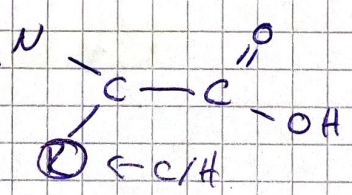
[#6]1[!#6]*1

c) an aromatische Systeme gebundene Nitro-Gruppe

[N+](=O)([O-])~a

↳ aromatisches Atom

d) Das erste Atom (C- Bindung H), das sich in einer Seitenkette einer α-Aminosäure befindet



R = C or H

[#6,#1;$([#6,#1]C(N)C(=O)O)]

α

[C,H;$([C,H]C(N)C(=O)O)]

BRUNNEN

GCI

5-1

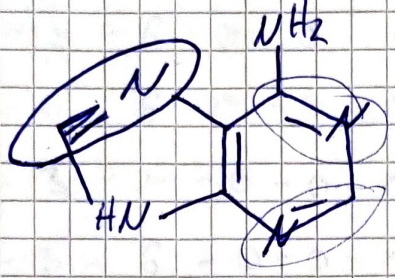
2)

aromatisch, Ring>0, aromatisch, nicht 4 Bindg, >3 Bindg

[c]-,:[c & !R0][n;!X4;X3]

<=> CC[n;X3]

4)



[#7; HO; #7=#6[#7&H1]] = #6

or

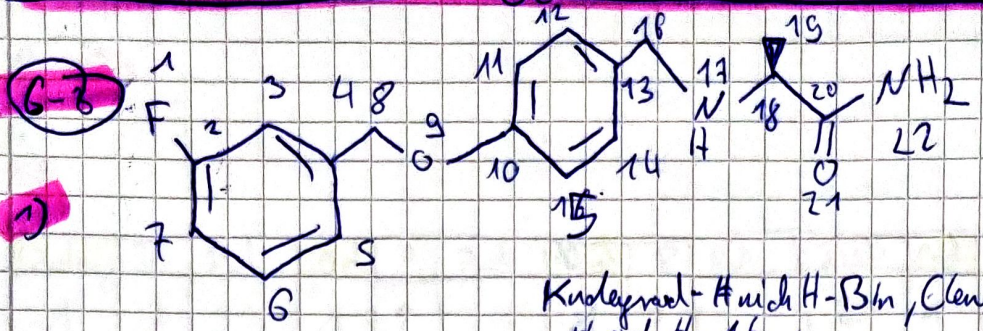
[#7; HO; r5] = #6

or

#7 = [#6; X2; !$(#6-#7=#6)]

BRUNNEN

GACI



Kudegraad: 4 mich H-Bin, Clement, Sgn (Lady), als (Lady, 4 geht H-Atome

1:	1-1-9-0-0-0	12:	2-3-6-0-0-1
2:	3-4-6-0-0-0	13:	3-4-6-0-0-0
3:	2-3-6-0-0-1	14:	2-3-6-0-0-1
4:	3-4-6-0-0-0	15:	2-3-6-0-0-1
5:	2-3-6-0-0-1	16:	2-2-6-0-0-2
6:	2-3-6-0-0-1	17:	2-2-7-0-0-1
7:	2-3-6-0-0-1	18:	3-3-6-0-0-1
8:	2-2-6-0-0-2	19:	1-1-6-0-0-3
9:	2-2-8-0-0-0	20:	3-4-6-0-0-0
10:	3-4-6-0-0-0	21:	1-2-8-0-0-0
11:	2-3-6-0-0-1	22:	1-1-7-0-0-2

Min over Eigenschaften

1 | 2 | 3 | 4 | 5 | 6 | 7 | 8

=> 19 | 22 | 1 | 21 | 8, 16 | 17 | 9 | 3, 5, 6, 7, 11, 12, 14, 15

9 | 10

| 18 | 2, 4, 10, 13, 20

BRUNNEN

GCI

(8-3)

1)

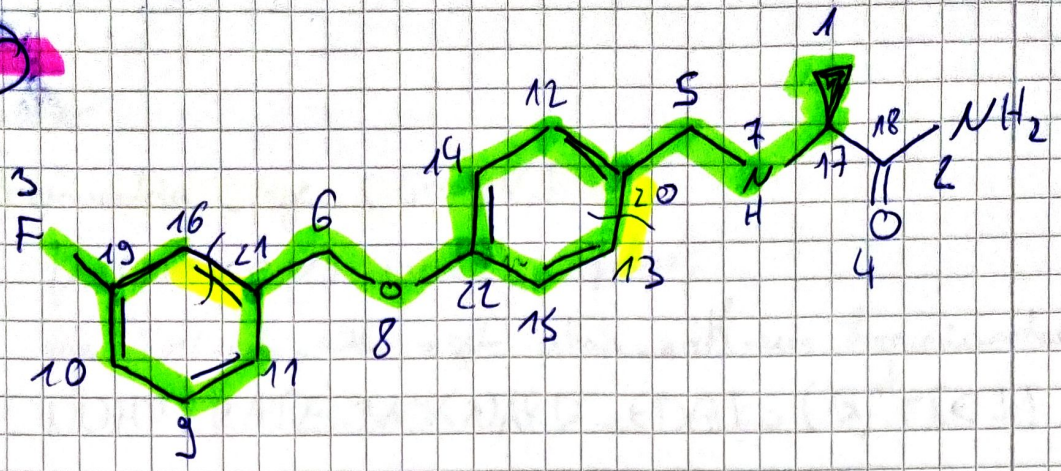
Atom	Ordnung	P	Prod	Ord	P	Prd	Ord	P	Prd	Tieß	Ord	P	Prd	Ord
19	1	2	23	1	2		1	2		2	1	2		1
22	2	3	29	2	3		2	3		4	2	3		2
1	3	5	29	3	5		3	5		6	3	5		3
21	4	7	29	4	7		4	7		8	4	7		4
8	5	11	493	6	13		6	13		12	6	13		6
16	5	11	377	5	11		5	11		10	5	11		5
17	6	13	253	7	17		7	17		14	7	17		7
9	7	17	319	8	19		8	19		16	8	19		8
3	8	19	841	11	31		14	43		28	15	47		16
5	8	19	551	10	29	1081	11	31		22	11	31		11
6	8	19	361	9	23		9	23		18	9	23		9
7	8	19	551	10	29	985	10	29		20	10	29		10
11	8	19	551	10	29	1537	13	41	2627	26	13	43	2701	14
12	8	19	551	10	29	1363	12	37	2501	23	12	37		12
14	8	19	551	10	29	1363	12	37	2501	24	13	41		13
15	8	19	551	10	29	1537	13	41	2627	26	14	43	2911	15
18	9	23	754	12	37		15	47		30	16	53		17
2	10	29	1805	14	43		17	59		34	18	61		19
4	10	29	3971	15	47	1687	19	67		38	20	71		21
10	10	29	6137	16	53		20	71		40	21	73		22
13	10	29	3971	15	47	9251	18	61		36	19	67		20
20	10	29	483	13	41		16	53		32	17	59		18

BRUNNEN

GC1

6-3

1)



3) Hauptkette / lu Chi = 15 / C18 H20 F N O2

/c1-17(7-5-20-12-14-22)
8(8-6-21-11-9-10-19(3)16-21)
15-13-20)18(2)4

Start bei min #B / min Order

/c1-17(7-5-20-12-14-22
(8-6-21-11-9-10-19(3)-16-21)
15-13-20)18(2)4

BRUNNEN

GAC1

6-3

4) Bezier

```
gen_connectivity_layer ( G=(V,E) )
    for (∀ v ∈ V) visit[v] = false;
    min_hor_min_order = get_datum_with_min_hor_min_order(V)
    NUMMERATE_BACKWARD_EDGES(G, int r[E])
    dfs_print('/c')
    dfs_visit(min_hor_min_order, NIL, r)

dfs_visit(datum v, datum pred, int& r)
    visit[v] = True
    print(v.CANON)
    nof_fwd = |v.adj| - (1 if parent != NIL else 0)   // Mid bad ap
    bads_worked = 0
    for (∀ (v,w) ∈ v.adj)
        if (r[(v,w)] > 0 and (v,w) != (v,parent)):   // Minus Backward Edges
            nof_fwd --
    for (∀ w ∈ v.adj):
        if (not visited[w] or (visited[w] && w != parent)):
            if (nof_fwd == 1): print('='); dfs_step(w)
            if (nof_fwd > 1):
                if (bads_worked == 0): print('('); dfs_step(w)
                if (bads+1 != nof_fwd): print(','); dfs_step(w)
                if (bads+1 == nof_fwd): print(')'); dfs_step(w)
                bads_worked ++

dfs_step(datum w)
    if(not visit[w]): dfs_visit(w)
    else: print(w.CANON)   // Ringschluss
```

Backward edges in edge for max from Ring!

v.adj.sel_min()

BRUNNEN

GdI

G-3

Start: Höhe min / Ordnungszahl min

2) Pseudocode Connection Layer InCh1

```
gen_connection_layer (G=(V,E))
    for (∀u∈V) visit[v] = false;
    min_hoer_min_order = get_atom_with_min_hoer_min_order (V)
    print('\c')
    dfs_visit (V, NIL    )
```

```
dfs_visit ( atom v , atom p )
    visit[v] = True
    print (v.V)          // print zimmer v. CANON
    nof_cov = 0
    ~~for (each w ∈ v.adj)~~ ← v.adj.sort_min_Ordernumber()
    for (each w ∈ v.adj)
        if ( not visit[w] ) nof_cov ++
    for (each w ∈ v.adj)
        if (not visit[w]):
            if nof_cov > 1 :
                print("("); dfs_visit(w); print(")");
            if nof_cov == 1:
                print("-"); dfs_visit(w);
            nof_cov --
        else if visit[w] and w != p :        // Ringschluss
            print (w)
```

BRUNNEN TH

GrCl

6-3

3) dfs-visit (atan v, atan p)

visit[v] = True

print (v. CANON)

nof_bonds = |v.adj| - (1 if parent != NIL else 0)

v.adj.sort-min-ordernumber ()

bonds_worked = 0

for (each w ∈ v.adj):

is_not_parent = ((w != p) && visit[w] = True)

if (not visited[w] && is_not_parent && not_visit)

if (nof_bonds == 1): print ("-"); (dfs_visit(w) if not visited[w] else print(w));

if (nof_bonds == 2):

if (bonds_worked = 0): print ('('); (dfs visit(w) if not visited[w] else print(w))

if (bonds_worked = 1): print (')'); (dfs_visit(w) if not visited[w] else print(w))

bonds_worked ++

if (nof_bonds > 2):

if (bonds_worked = 0): print ('('); (dfs_visit(w) if not visit[w] else print(w))

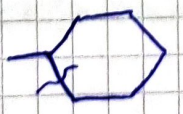
if (bonds_worked + 1 != nof_bonds):

print (','); (dfs_visit(w) if not visit[w] else print(w))

if (bonds_worked + 1 == nof_bonds):

print (')'); (dfs_visit(w) if not visit[w] else print(w))

bonds_worked ++

→ Problem  Ringe werden nicht korrekt dargestellt.

Lösung: Ringe vorher klassifizieren mit backward-up/down Kanten richtig reagieren!

BRUNNEN